Full Stack Development with MERN

Project Documentation

1. Introduction

- **Project Title:** ShopEZ Ecommerce Application
- **Team Members:** Aditya Kamble (Individual)
- **GitHub Repository:** https://github.com/Aditya-Kambale/ShopEZ_MERN_Stack

2. Project Overview

• Purpose:

ShopEZ Application is a modern, responsive full-stack e-commerce web platform engineered using the MERN stack (MongoDB, Express.js, React 18, Node.js). Designed for everyday online retail, it provides consumers with a frictionless shopping journey—from product catalog exploration, keyword search, and category filtering to persistent shopping cart synchronization, transparent reviews, and Cash on Delivery (COD) checkout. Simultaneously, it equips store merchants and administrators with a dedicated Seller Central portal featuring real-time inventory tracking, inline price/stock updates without page reloads, automated low-stock warnings (< 10 units), and gross revenue analytics.

• Features:

- **User Authentication:** Registration, secure login, stateless JWT bearer token authentication, role-based access control (`user` / `admin`), and client/server protected route wrappers.
- **Interactive Storefront:** Category filtering (Electronics, Clothing, Footwear, Home, Books), instant regex search, and multi-parameter price sorting.
- **Product Detail & Customer Reviews:** High-resolution product images, discount tags, dynamic stock indicators, and customer rating/review submission with 1–5 stars.
- **Persistent Shopping Cart:** Real-time cart calculations, quantity increment/decrement, and line-item subtotals.
- **Streamlined Checkout & Order Tracking:** Cash on Delivery (COD) checkout, automatic stock deduction, and real-time status tracking (`Processing` → `Shipped` → `Delivered`).
- **Seller Central & Operations:** Dedicated `/admin/sell` dashboard with sales summaries, inline editable table cells for prices and stock counts, and automated low-stock warnings (< 10 units).
- **Admin Dashboard:** Centralized hub displaying gross revenue analytics, order volume, total product listings, and registered customer governance.

3. Architecture

The application follows a strictly decoupled 3-tier client/server architecture. The React single-page application communicates asynchronously with an Express.js backend via RESTful JSON endpoints. Requests are validated through custom middleware before reaching controllers that interact with MongoDB via Mongoose ODM schemas.

ShopEZ Markets — System Architecture

A secure, scalable, and modular 3-tier full-stack architecture for modern e-commerce operations.

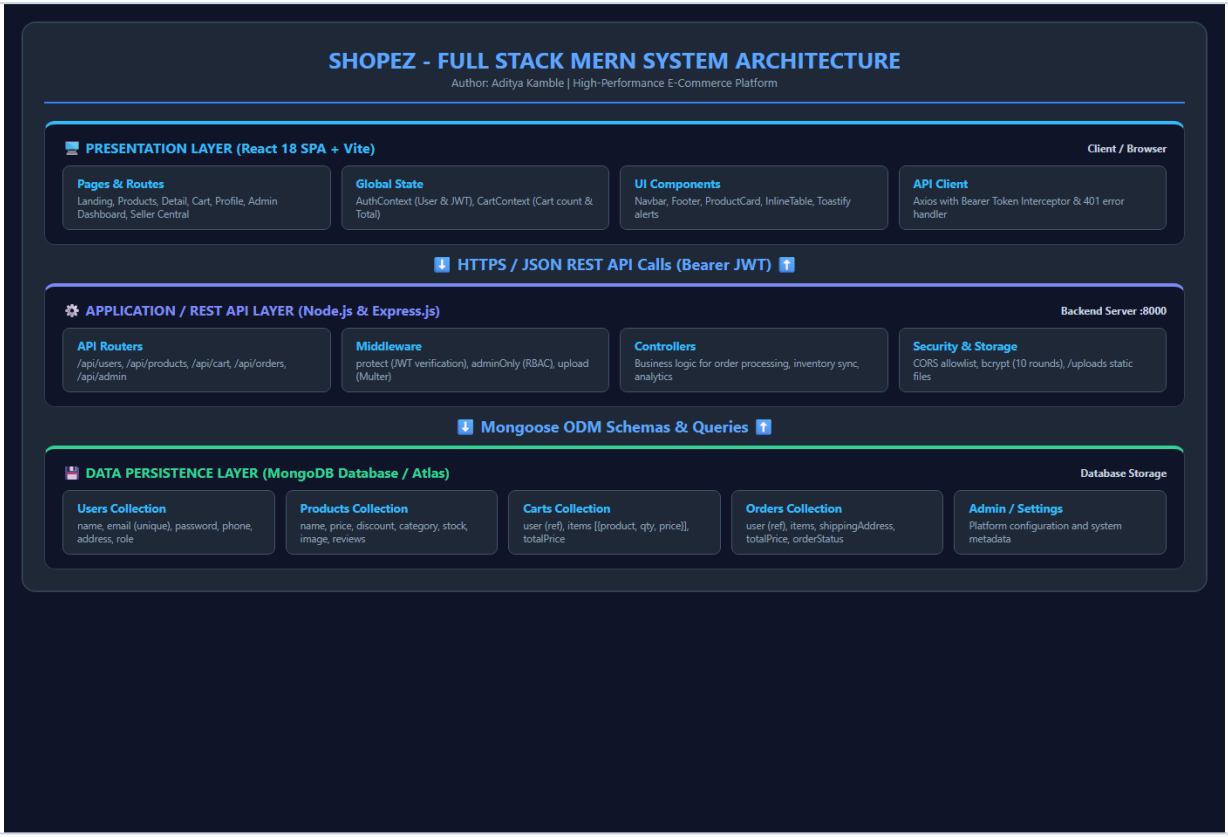


Figure 1: Full-Stack MERN System Architecture (Client, API Server & Database Layers)

Frontend Architecture (React 18 + Vite)

The frontend is built with React 18 and Vite for near-instant build times and Hot Module Replacement (HMR). Routing is structured through React Router DOM (v7) with declarative ProtectedRoute wrappers. Global application state is handled using the React Context API (AuthContext, CartContext, and Axios HTTP client with Bearer token interceptor).

Backend Architecture & Database Schema

The backend REST API server follows the classic MVC pattern with routes, middleware, controllers, and Mongoose ODM schemas.



Figure 2: MongoDB Entity Relationship (ER) Schema & Collection Design

4. Setup Instructions & Prerequisites

- Node.js (v16+ or v18+ recommended) and npm
- MongoDB locally running on `localhost:27017` or MongoDB Atlas URI
- A modern web browser

- **Environment Variables:**

File	Variable	Purpose & Default Value
`Server/.env`	`PORT`	Express server port (default `8000`)
`Server/.env`	`MONGO_URI`	MongoDB connection string (e.g., `mongodb://localhost:27017/shopez`)
`Server/.env`	`JWT_SECRET`	Secret cryptographic key used for signing JWTs
`Client/.env`	`VITE_API_URL`	Backend API base URL (default `http://localhost:8000/api`)

5. Folder Structure

```
ShopEZ E-commerce Application/ ├── Documentation/ # Project Documentation.pdf, Project_Documentation.md, diagrams/ ├── Phase Wise/ # Complete 5-phase engineering deliverables ├── Client/ # React 18 frontend (Vite) └── Server/ # Node.js & Express.js backend
```

6. Running the Application & API Endpoints

```
cd Server && npm install && node seed.js && npm run dev cd ../Client && npm install && npm run dev
```

7. API Documentation Table

Method	Endpoint	Access	Description
POST	/api/users/register	Public	Register a new user: name, email, password
POST	/api/users/login	Public	Authenticate user and return JWT bearer token
GET	/api/products	Public	Fetch catalog (search, category, sort)
POST	/api/cart/add	User	Add product item to cart
POST	/api/orders	User	Place new order with shipping address & COD
GET	/api/admin/dashboard	Admin	Fetch aggregate metrics: revenue, orders, products, users

Sample API Payloads

1. Registration (POST /api/users/register)

```
POST /api/users/register { "name": "Aditya Kamble", "email": "aditya.kamble@example.com", "password": "SecurePassword123" } // Response 201 Created: { "success": true, "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", "user": { "name": "Aditya Kamble", "role": "user" } }
```

2. Order Placement (POST /api/orders)

```
POST /api/orders Authorization: Bearer <JWT> { "items": [{ "product": "64f1c9d...", "name": "Wireless Headphones", "price": 2499, "quantity": 1 }], "shippingAddress": { "address": "45 Tech Park Avenue", "city": "Pune", "state": "MH", "zipCode": "411045", "country": "India" }, "paymentMethod": "COD", "totalPrice": 2499 }
```

8. Authentication Architecture & JWT Flow

Stateless authentication using JSON Web Tokens with bcryptjs password hashing and role-based route guards.

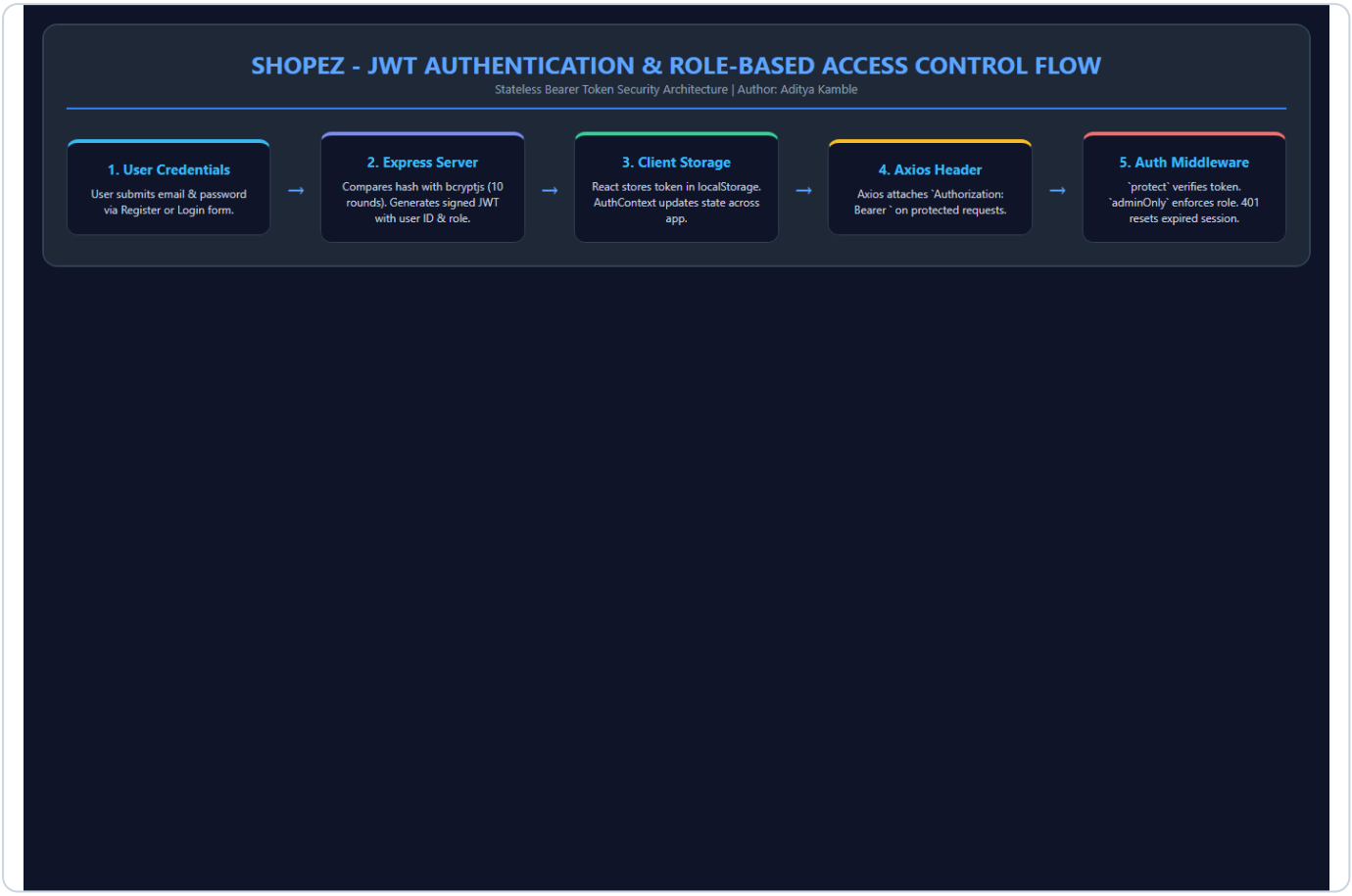


Figure 3: ShopEZ JWT Authentication, Token Persistence & Role-Based Guard Flow

9. Testing Strategy & Quality Assurance

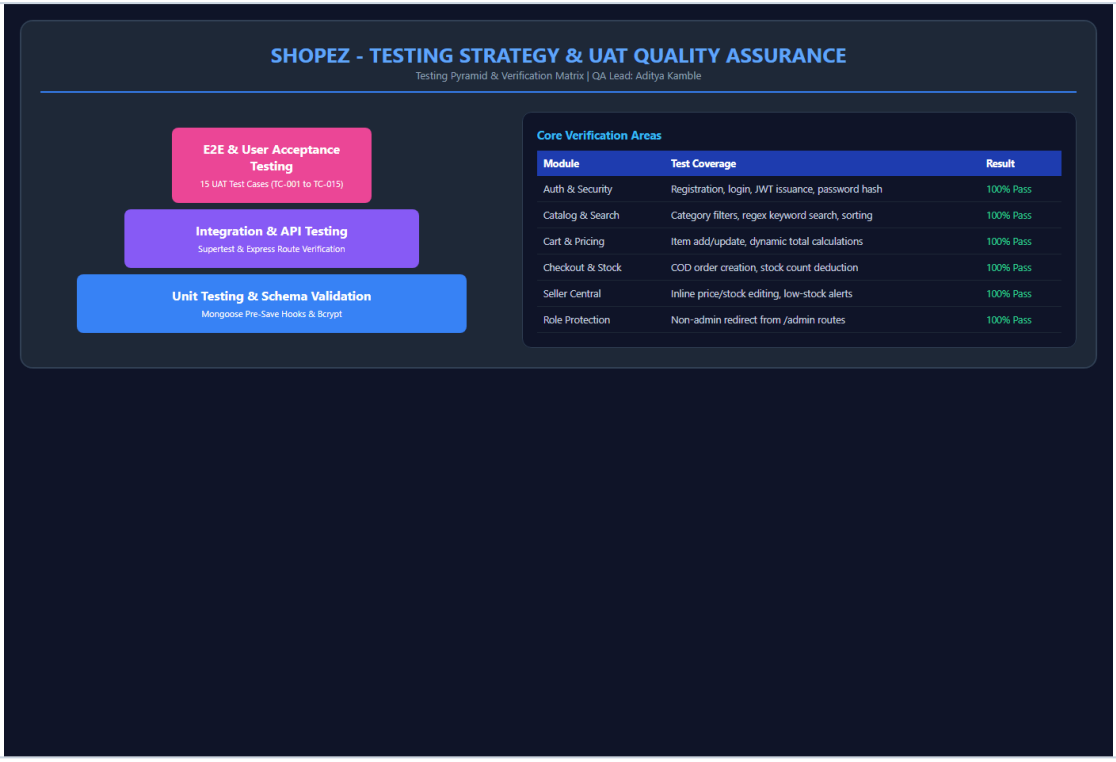


Figure 4: Testing Pyramid and Verification Areas for ShopEZ

Test ID	Scenario	Verification Result	Status
TC-001	User Registration	Account created, JWT signed and returned	PASS
TC-002	User Login	Token stored in localStorage, navbar updates	PASS
TC-006	Shopping Cart Update	Dynamic subtotal recalculation confirmed	PASS
TC-008	Checkout (COD)	Order created, stock decremented in MongoDB	PASS
TC-012	Inline Inventory Edit	Price and stock updated immediately via PUT request	PASS

10. User Interface — Home / Landing Page

Storefront landing page featuring hero banner, category pills, and featured products.

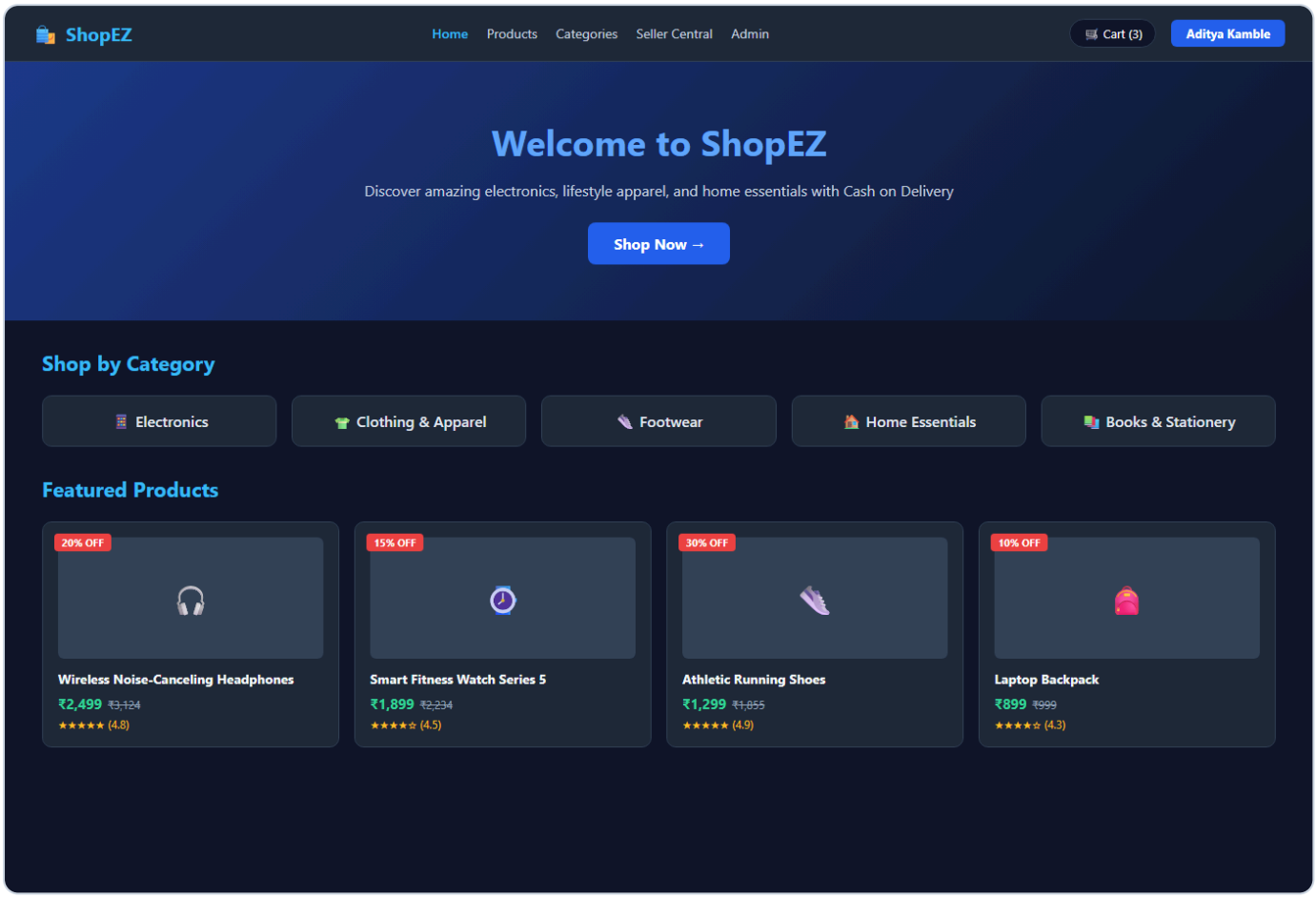


Figure 5: ShopEZ Storefront Landing Page with Hero Banner & Featured Product Grid

User Interface — Product Catalog & Filtering

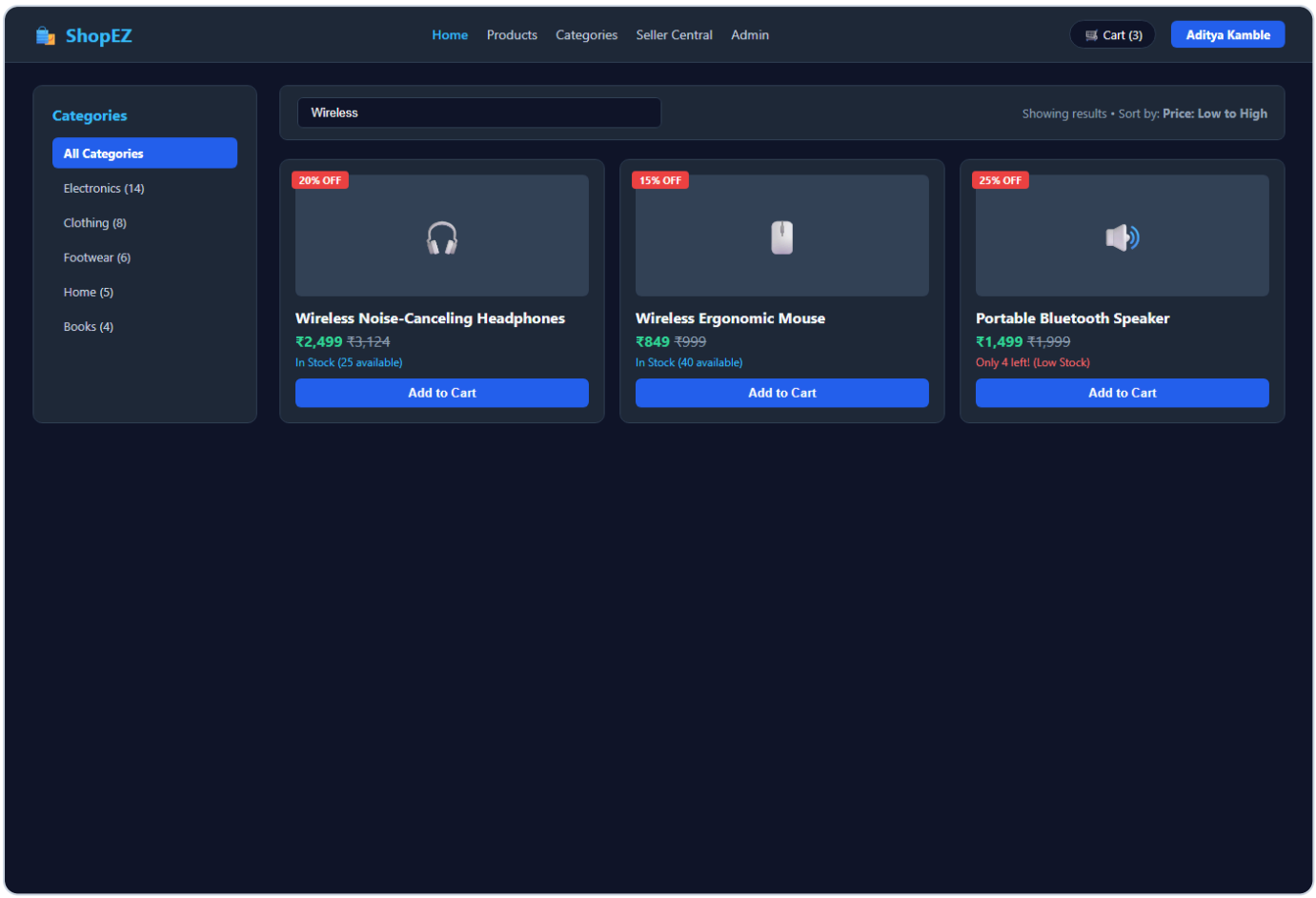


Figure 6: Interactive Product Catalog with Search Bar & Category Navigation

User Interface — Shopping Cart & Price Recalculation

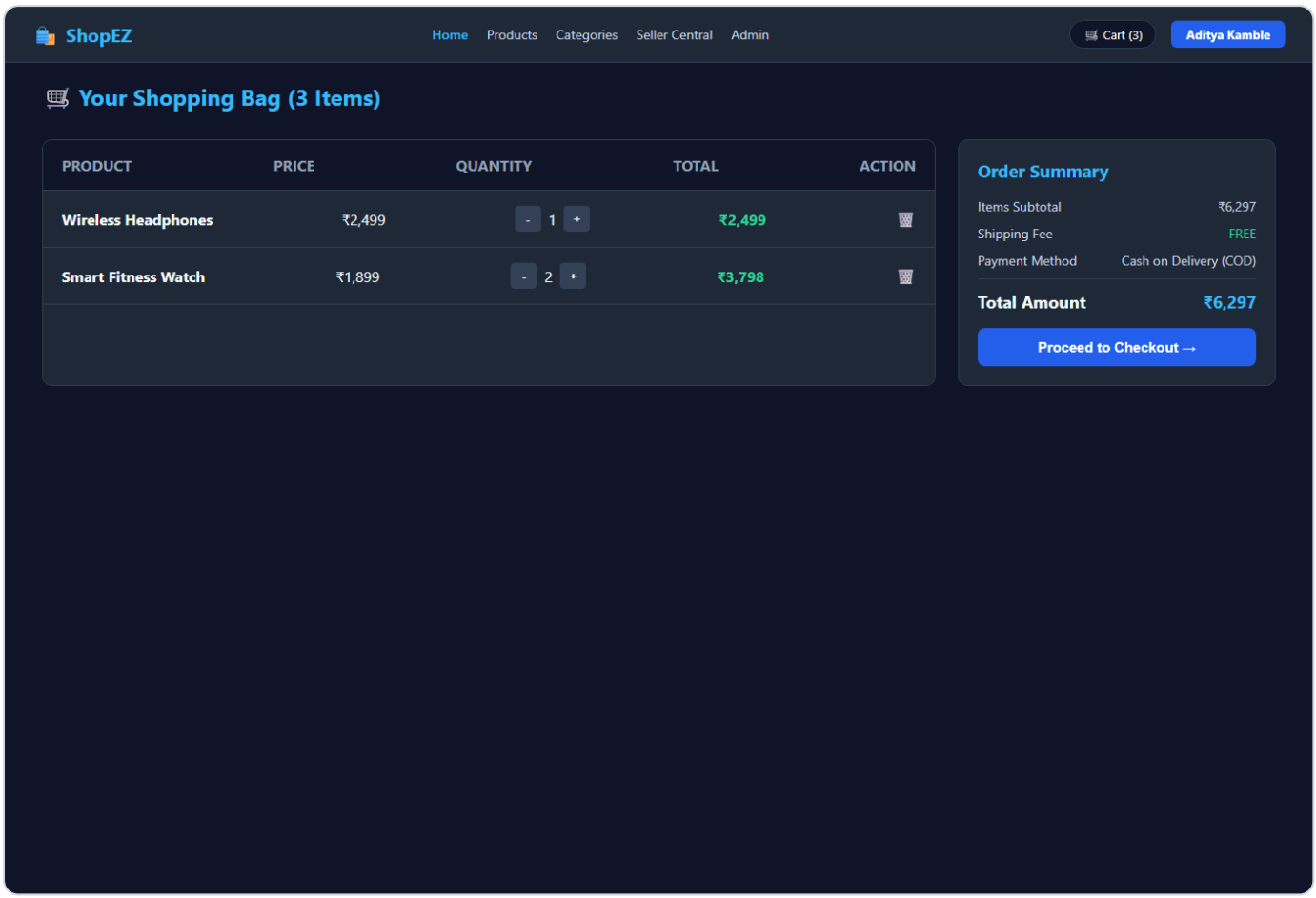


Figure 7: Shopping Bag Interface with Dynamic Line-Item & Total Price Recalculations

User Interface — User Profile & Order History

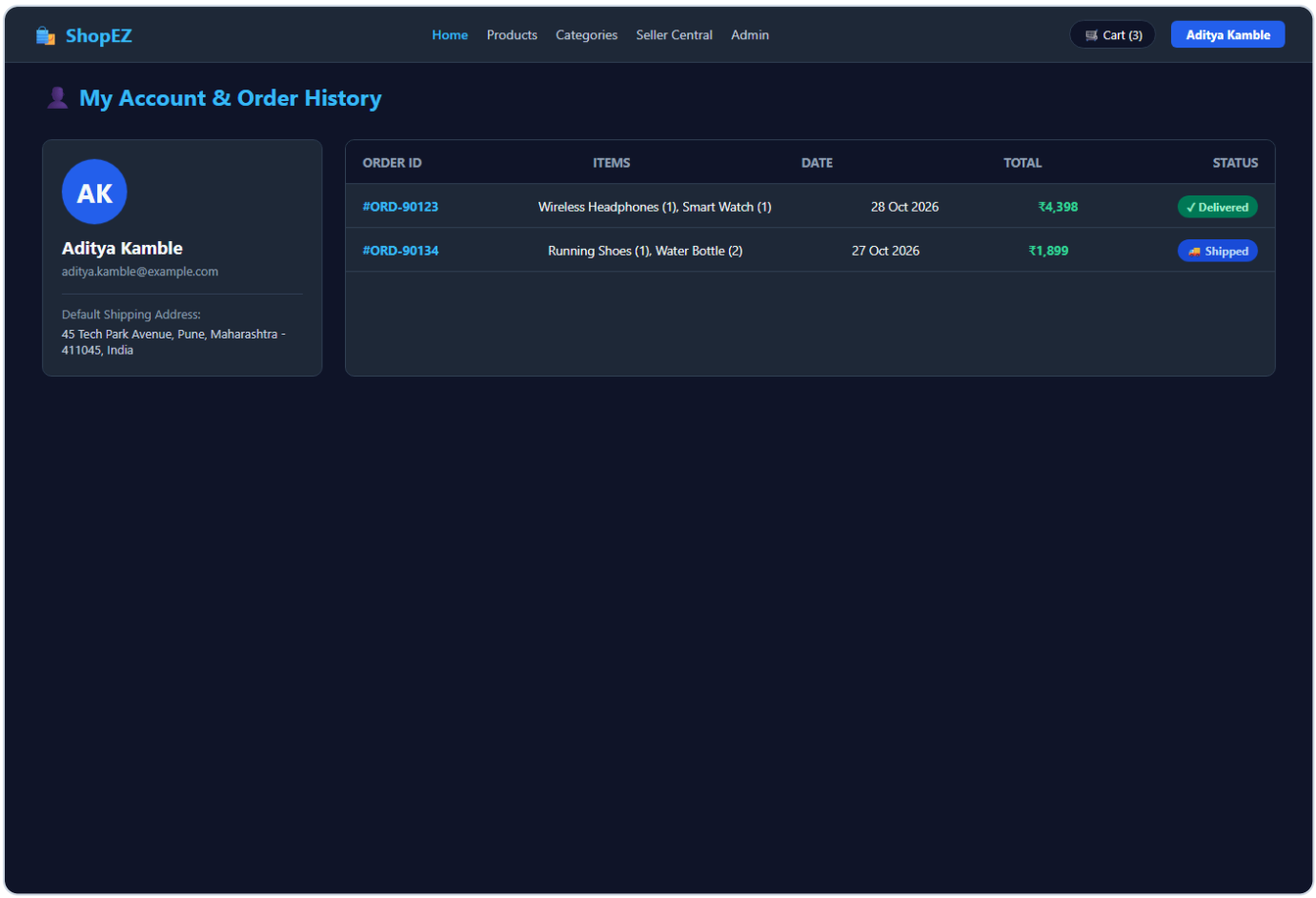


Figure 8: Customer Profile Dashboard with Real-Time Order History & Tracking

User Interface — Seller Central Dashboard (/admin/sell)

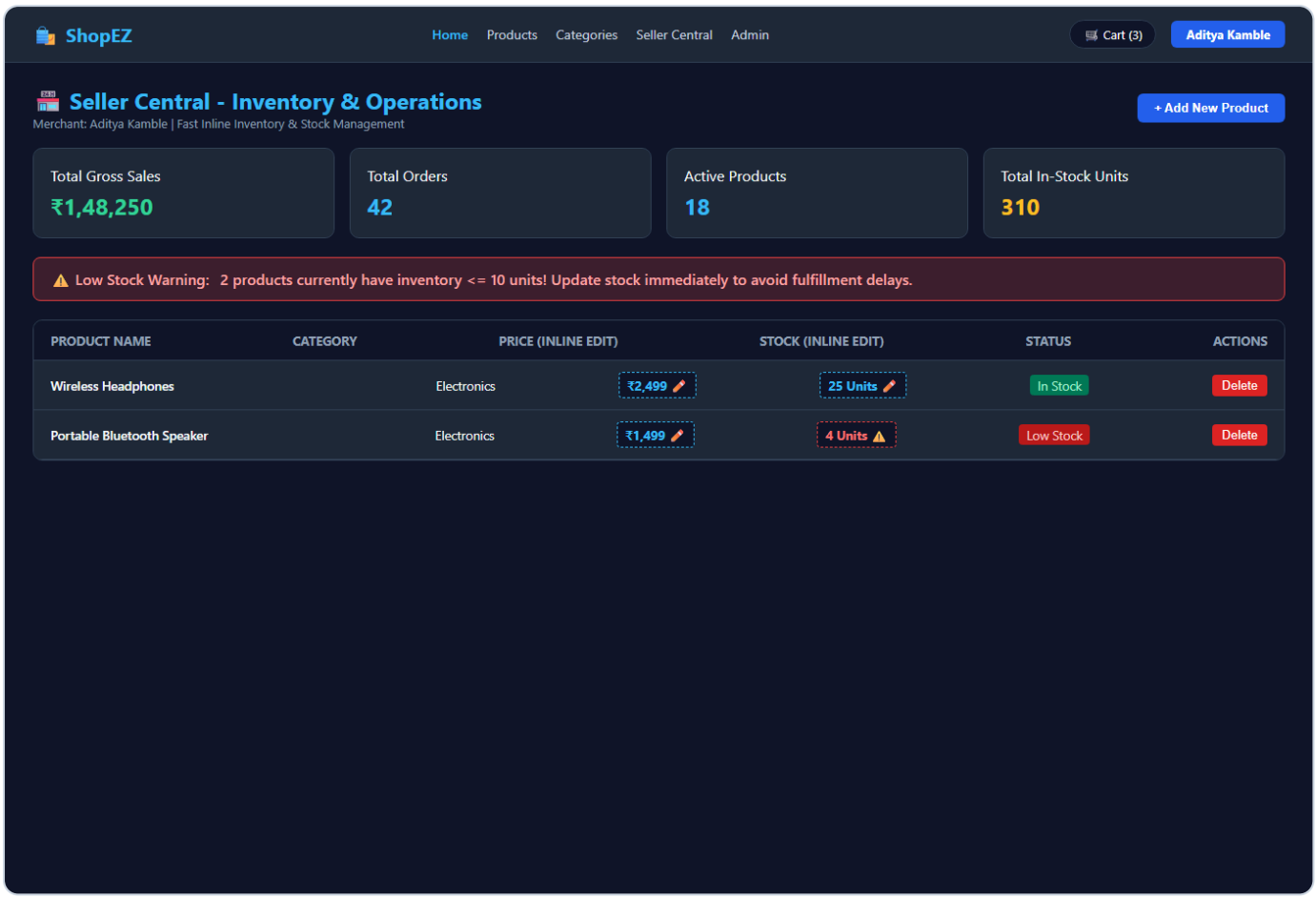


Figure 9: Seller Central Dashboard with Inline Price/Stock Editing & Low-Stock Alerts

Data Flow Diagrams (DFD Level 0, 1 & 2)

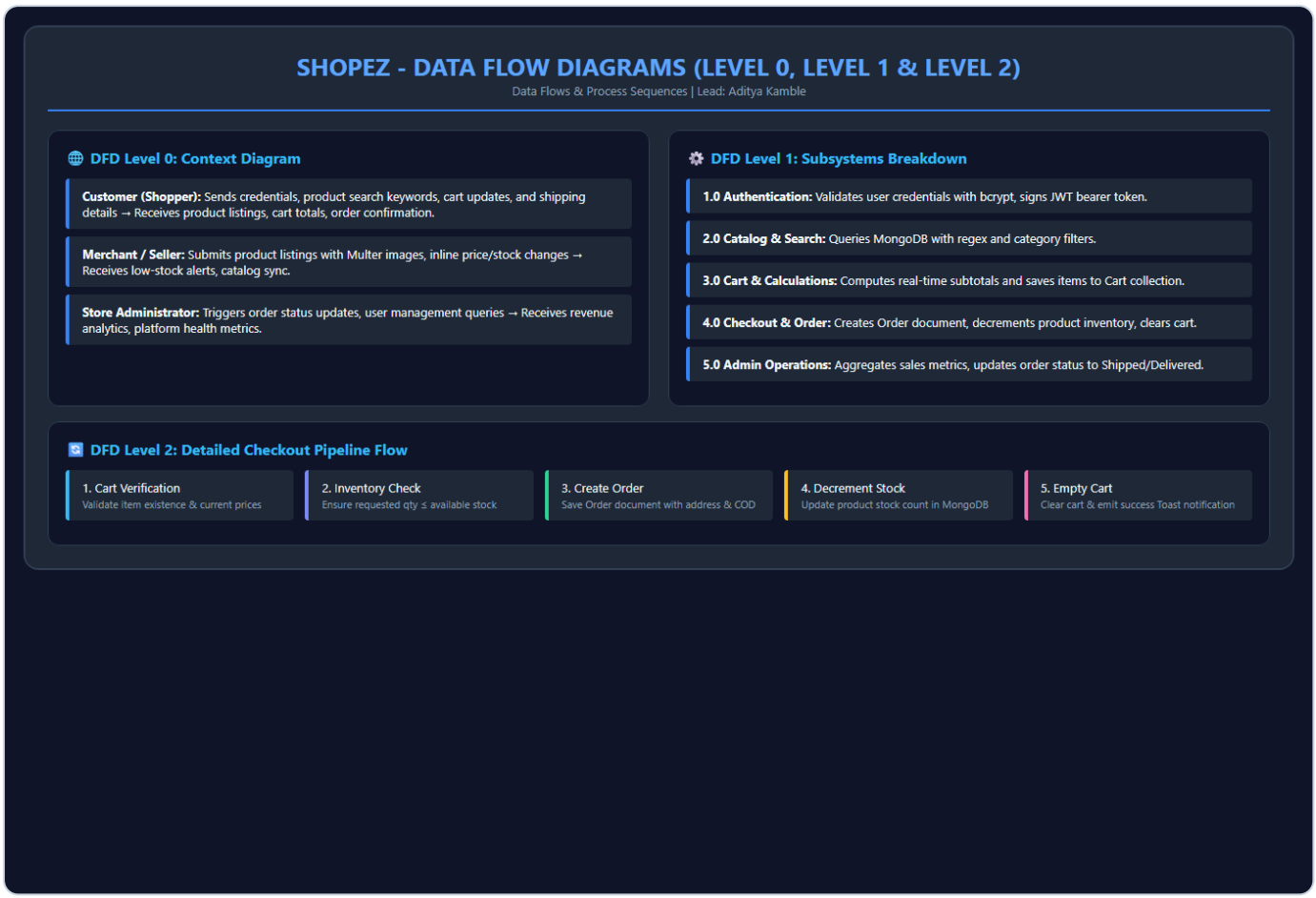


Figure 10: Multi-Level Data Flow Diagrams for ShopEZ Subsystems & Checkout Pipeline

Ideation Phase — Empathy Map & Problem Statement



Figure 11: Empathy Map Canvas for Online Consumers and Store Merchants

Ideation Phase — Action Priority Matrix (Impact vs Effort)

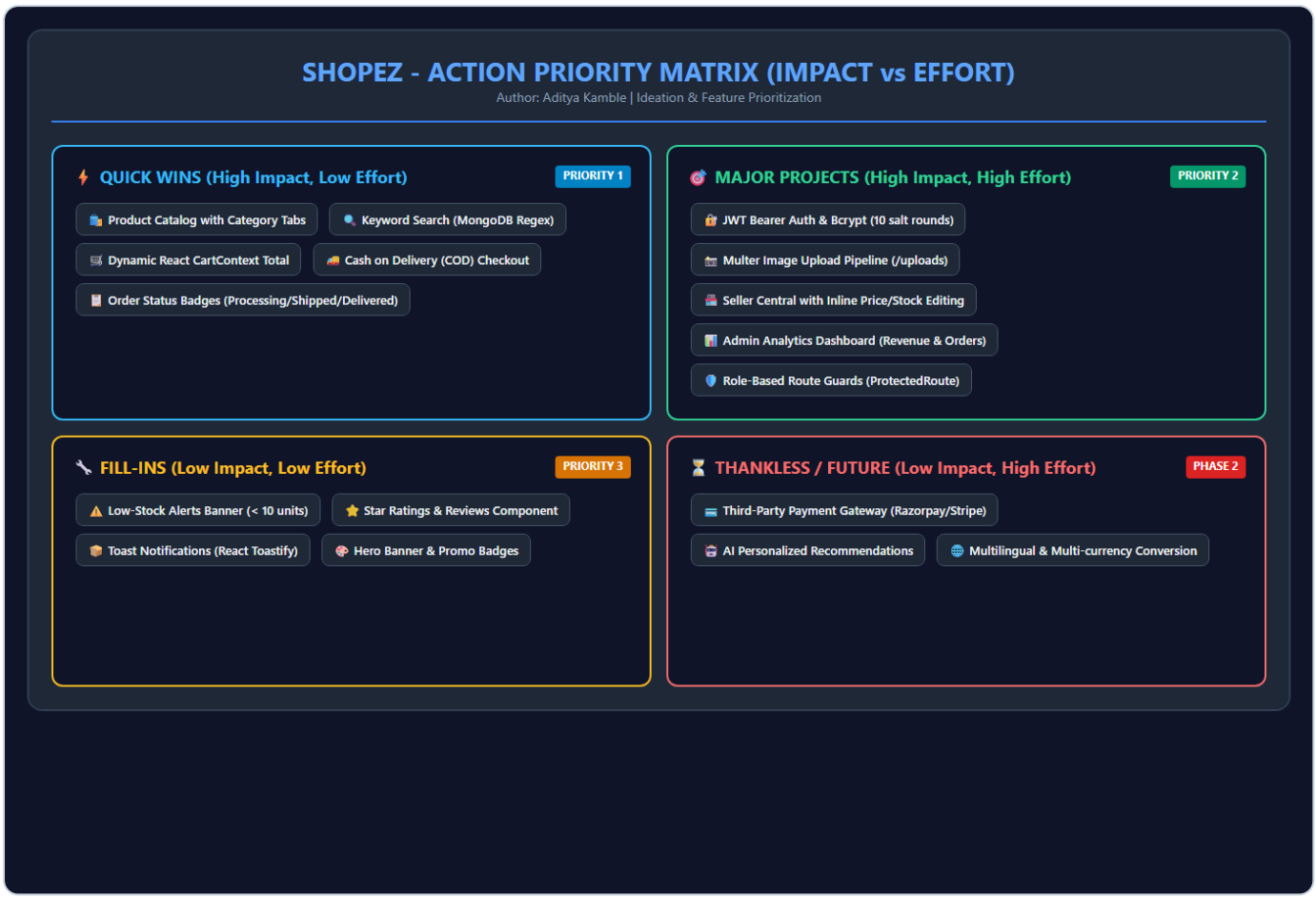


Figure 12: Action Priority Matrix (Quick Wins, Major Projects, Fill-Ins & Future Iterations)

11. Known Limitations & Future Enhancements

- **Cloudinary Integration:** Cloud storage integration for persistent image hosting in serverless deployments.
- **Payment Gateways:** Integration with Stripe & Razorpay for online cards and UPI.
- **Email & SMS:** Automated order dispatch receipts via Nodemailer.

12. Project Links & Verification

- **Author:** Aditya Kamble (Individual)
- **GitHub Repository:** https://github.com/Aditya-Kambale/ShopEZ_MERN_Stack
- **Documentation Status:** Approved & Verified (17 Pages with Architecture & UI Diagrams)